

MemoryGraph AI: A Dynamic, Context-Aware Hybrid Knowledge Graph Framework for Documents with Comparative Vector Database Analysis

SEE

Project Guide:

Dr.Shouvik Chakraborty

Assistant Professor

School of Computer Science and Engineering

RV University, Bangalore

Team members

1RVU22CSE020, Amrutha M

1RVU22CSE036 - Bhargava Srinivasan

1RVU22CSE129 - Renu Bojja

Outlines

1. Introduction
2. Problem statement
3. Summary of Literature survey
4. Objectives and Expected outcomes
5. Proposed Methodology
6. System Architecture
7. Algorithms and Implementation
8. Testing Integration
9. Results and Discussion
10. Publication Status
11. Conclusion & Future work
12. References

1.Introduction

Some of the key motivations behind developing MemoryGraph AI include:

- Limitations of keyword and vector based retrieval
- Poor semantic relationship understanding
- Lack of persistent knowledge representation
- Need for context aware multi document retrieval
- Cross document contextual information retrieval
- Domain independent document intelligence framework

MemoryGraph AI: A hybrid framework integrating knowledge graphs and embedding-based retrieval for context-aware document retrieval.

2.Problem statement

Traditional document retrieval systems often fail to provide context-aware answers across multiple documents because they lack structured semantic memory. MemoryGraph AI overcomes this limitation by building a persistent multi-document knowledge graph that enables cross-document reasoning and more accurate context-aware retrieval.

3.Summary of Literature Survey

Paper Details	Objectives of Paper	Key Contribution	Research Gap
Graph-Anchored Knowledge Indexing for Retrieval-Augmented Generation - Zhenghao Liu ¹ , Mingyan Wu et. al.	- Introduces GraphAnchor, where a knowledge graph is incrementally updated during iterative retrieval - Graph acts as a knowledge anchor, guiding the LLM's attention toward relevant entities	Graph-based indexing improves multi-hop retrieval reasoning.	graph quality strongly depends on the LLM's ability to extract accurate entities lacks richer graph semantics or hybrid indexing with other retrieval methods
Graph neural networks: A review of methods and applications - Jie Zhou, Ganqu Cui et. al.	Survey of GNN architectures and graph learning techniques like MegaRAG, MMKG etc.	Provides overview of graph representation learning methods and architectures.	document segmentation can still cause missing relationships Static graphs limits adaptability for dynamic document collections

Summary of Literature Survey

MegaRAG: Multimodal Knowledge Graph-Based Retrieval Augmented Generation - Chi-Hsiang Hsiao, Yi-Cheng Wang et. al.	Multimodal Knowledge Graph Construction from Document Content unified embedding framework that maps entities, relations, and document pages into a shared vector space	Enables hybrid retrieval using graph and document pages.	Limited Dynamic Updating of the Multimodal Knowledge Graph Lack of Comparative Analysis of Vector Database Architectures
KG ² RAG: Knowledge Graph-Guided Retrieval Augmented Generation - Xiangrong Zhu, Yuexiang Xie et. al.	Knowledge Graph-guided chunk expansion KG-based context organization mechanism Triplet extraction and chunk-KG association	Improves context organization and multi-hop retrieval.	The KG is built during offline preprocessing, which limits dynamic updates No comparative analysis of vector databases

Summary of Literature Survey

GeAR: Graph-Enhanced Agent for Retrieval-Augmented Generation - Zhili Shen, Chenxin Diao et. al.	Introduces SyncGE, a graph-based retriever that expands search paths using semantic triples linking related passages unified embedding framework that maps entities, relations, and document pages into a shared vector space	Enables hybrid retrieval using graph and document pages.	Limited Dynamic Updating of the Multimodal Knowledge Graph Lack of Comparative Analysis of Vector Database Architectures
Knowledge Graph Construction: Extraction, Learning, and Evaluation - Seungmin Choi, Yuchul Jung et. al.	Surveys on how KGs are built and improved Exploration of KG pipelines including extraction, learning, and evaluation.	KG evaluation framework for quality assessment. Intrinsic evaluation, extrinsic evaluation and qualitative evaluation	KG is mostly static and not context-aware Do not effectively combine with vector databases

Summary of Literature Survey

Docs2KG: Unified Knowledge Graph Construction from Heterogeneous Documents Assisted by Large Language Models - Qiang Sun, Yuanyi Luo	Unified Knowledge Graph from heterogeneous documents Dual-path document processing architecture	Focuses on LLM-assisted multimodal extraction from documents.	Lack of comparative analysis with vector-based retrieval systems Limited hybrid retrieval evaluation
AutoKG: Constructing Virtual Knowledge Graphs from Unstructured Documents for Question Answering	Automatic KG construction from unstructured documents Context-aware entity linking using BERT embeddings Multi-hop graph traversal for QA retrieval	Enables multi-hop QA using automatically generated graphs.	Does not explicitly address dynamic integration and relationship discovery across multiple uploaded documents

Summary of Literature Survey

RAKG: Document-level Retrieval Augmented Knowledge Graph Construction - Hairong Zhang, Jiaheng Si	Retrieval-Augmented Knowledge Graph Construction Progressive Knowledge Extraction using Pre-Entities RAG-based Evaluation for Graph Quality	Improves entity extraction accuracy using contextual retrieval.	Lack of Persistent Knowledge Graph Memory System cannot perform cross-document contextual retrieval
A Comprehensive Survey on Automatic Knowledge Graph Construction - <u>Lingfeng Zhong</u> , <u>Jia Wu</u>	Unified Taxonomy for Knowledge Graph Construction Integration of Neural Architectures for Graph Representation Learning	Introduces taxonomy for knowledge graph construction methods.	Limited Integration Between Knowledge Graphs and Retrieval Systems

Summary of Literature Survey

KG-BERT: BERT for Knowledge Graph Completion - Liang Yao, Chengsheng Mao	Applying Pre-trained Language Models to Knowledge Graph Completion Triple Representation as Textual Sequences Context-Aware Entity and Relation Representation	Uses BERT to model KG triples as textual sequences.	Lack of Knowledge Graph Construction from Documents Absence of Hybrid Retrieval or Graph-based Contextual Retrieval
HybridRAG: Integrating Knowledge Graphs and Vector Retrieval Augmented Generation for Efficient Information Extraction - Bhaskarjit Sarmah, Benika Hall et. al.	Hybrid Retrieval Architecture LLM-based Knowledge Graph Construction Multi-Metric RAG Evaluation Framework	Demonstrates hybrid retrieval architecture for QA systems.	Trained on financial data limits the generalizability and validation of the approach across diverse document domains.

Summary of Literature Survey

Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks - Patrick Lewis, Ethan Perez	Introduces a hybrid retrieval + generation architecture for knowledge-based reasoning. Demonstrates how vector embeddings enable semantic document retrieval.	Introduces RAG; Combines dense retrieval with transformer-based generation.	Lack of Structured Knowledge Representation Limited Multi-Hop Reasoning
Large Language Models Meet Knowledge Graphs for Question Answering: Synthesis and Opportunities - Chuangtao Ma, Yongrui Chen et. al.	Provides a systematic taxonomy for integrating LLMs with Knowledge Graphs. Demonstrates how knowledge graphs help reduce hallucination problems in language models	Discusses architectures combining graph embeddings and transformer models for reasoning tasks.	Weak Integration with Vector Retrieval Context Awareness Issues

Summary of Literature Survey

GNN-RAG: Graph Neural Retrieval for Large Language Model Reasoning - Costas Mavromatis, George Karypis et. al.	Enables multi-hop reasoning over knowledge graphs. Combines graph neural networks with LLMs. Demonstrates improved performance on knowledge-intensive QA tasks	Introduces GNN-RAG, a framework that combines Graph Neural Networks (GNNs) with RAG to improve reasoning capabilities	Computational Complexity Dependence on Knowledge Graph Quality
DO-RAG: A Domain-Specific QA Framework Using Knowledge Graph-Enhanced Retrieval-Augmented Generation - David Osei Opoku, Ming Sheng et. al.	Introduces a hybrid graph + vector retrieval architecture Provides hallucination mitigation using grounded graph reasoning.	Introduces DO-RAG, that integrates knowledge graphs with vector-based retrieval to improve domain-specific question answering systems	Limited Generalization Evaluation Domain Dependency

Summary of Literature Survey

Knowledge Graph-Guided Retrieval Augmented Generation - Xiangrong Zhu, Yuexiang Xie	Uses graph relationships to expand retrieved context beyond initial semantic matches Combines vector retrieval with graph traversal mechanisms	Improves coherence and diversity of retrieved information for question answering.	Limited Dynamic Knowledge Updates Scalability Issues for Large Document Graphs
E2GraphRAG: Streamlining Graph-based RAG for High Efficiency and Effectiveness - Yibo Zhao, Jiapeng Zhu	Combines hierarchical tree with an entity graph Graph-aware adaptive retrieval mechanism that automatically switches between local and global retrieval modes	Efficient Knowledge Graph Construction without use of LLMs	Lack of Cross-Document Knowledge Graph Integration Absence of Graph-Based Knowledge Memory

4.Objectives and Expected Outcomes

Objectives

- Identify limitations of existing retrieval systems
- Develop a hybrid knowledge graph and embedding retrieval framework
- Enable multi-document entity and relationship extraction
- Evaluate system performance through experiments

Expected Outcomes

- Development of a hybrid document retrieval system (frontend + backend)
- Construction of a multi-document knowledge graph
- Implementation of context-aware retrieval using graph and embeddings
- Research paper publication based on the proposed framework

5. Proposed Methodology

The proposed MemoryGraph AI framework:

1. Comparative Analysis of Retrieval Approaches

- Implementation of two retrieval pipelines: Vector-based retrieval and Hybrid (Knowledge Graph + Embeddings)
- Vector Retrieval: SentenceTransformer embeddings indexed using FAISS
- Hybrid Retrieval: Combines vector search with knowledge graph traversal
- Evaluation using Precision, Recall, F1-score, Coverage, Contextual Accuracy, and Query Latency
- Hybrid approach selected based on comparative performance analysis

5. Proposed Methodology

2. Document Ingestion and Preprocessing

- Input documents (PDFs) are ingested, processed and text is extracted using PyMuPDF
- Extracted content cleaned and segmented into text chunks

3. Entity and Relationship Extraction

- Identification of key entities and semantic relationships
- Extracted elements used to build structured knowledge representation

4. Multi-Document Knowledge Graph Construction

- Extracted entities and relationships stored in Neo4j
- Entities → Nodes, Relationships → Edges
- Graph populated using Cypher queries via MemoryGraphBuilder module
- Neo4j advantage: index-free adjacency enables $O(1)$ neighbor traversal and efficient multi-hop queries

5. Proposed Methodology

- Enables entity linking across documents
- Similarity computed using SentenceTransformer (all-MiniLM-L6-v2) embeddings
- Cosine similarity threshold used to merge entities
- Similar entities linked to existing nodes, otherwise new nodes created
- Unrelated documents remain as independent subgraphs

5. Embedding Generation and Vector Indexing

- Entity names converted to embeddings using SentenceTransformer (all-MiniLM-L6-v2)
- 384-dimensional embeddings generated for each entity
- Embeddings stored as node properties in Neo4j
- Neo4j vector index created for cosine similarity ANN search

5. Proposed Methodology

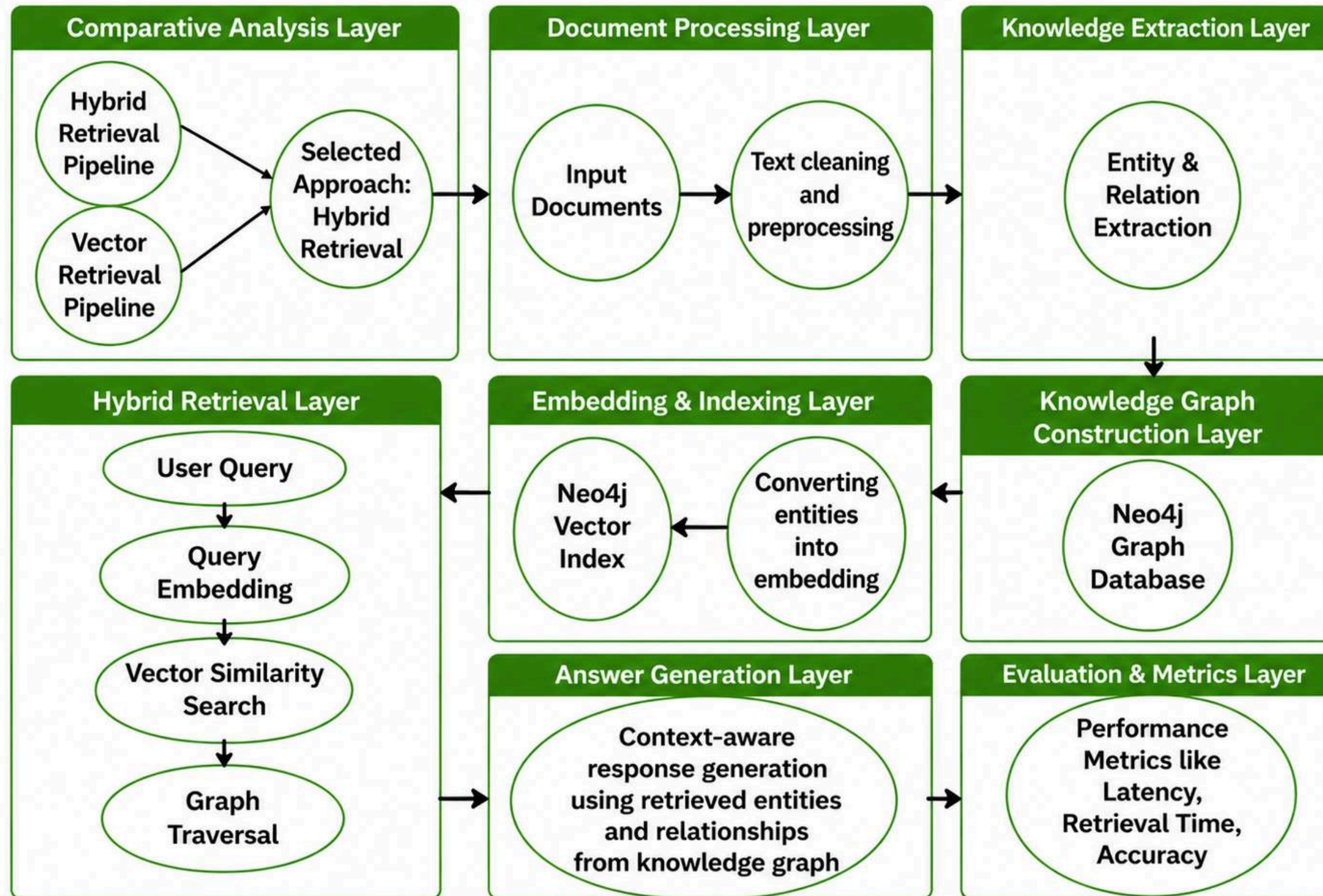
6. Hybrid Retrieval Mechanism

- User query converted into embedding using SentenceTransformer (all-MiniLM-L6-v2)
- Neo4j vector similarity search retrieves the most relevant nodes
- Cypher graph traversal explores connected nodes and relationships
- Enables multi-hop reasoning across related entities
- Combines semantic similarity + graph context retrieval

7. Answer Generation

- Retrieved graph context passed to the LLM
- LLaMA-3.1-8B-Instant synthesizes the retrieved information
- Generates a context-aware final response

6.System Architecture



7. Algorithms and Implementation

1. Comparision Module (run_comparison.py)

- Purpose: Evaluate and compare Hybrid RAG and Vector RAG
- LLM Judge → Scores answer & context quality
- Key Metrics used:
 1. Retrieval quality metrics
 2. LLM-as-Judge evaluation
 3. Context quality analysis
- Output:
 - Ranking metrics + LLM evaluation scores
 - Performance comparison (Hybrid vs Vector)

Algorithm / Pseudocode

INITIALIZE graph, embedder, and LLM
FOR each query: retrieve using Graph RAG (traversal) and Vector RAG (similarity)
Generate answers using LLM for both approaches
Evaluate using ranking metrics and LLM-as-judge (answer + context)
AGGREGATE results and compare Hybrid vs Vector performance

7. Algorithms and Implementation

2. Information Extraction (extraction.py)

- Purpose: Extract entities and relationships from text using LLM (Groq API).
- Key Functions:
 1. `chunk_text()` → Split large text into manageable chunks.
 2. `extract()` → Extract Knowledge Graph entities and relationships from chunks.
- Output: Dictionary with entities and relationships.

Algorithm / Pseudocode

Information Extraction:

FOR each document in the list:

Split document into chunks

FOR each chunk (up to max_chunks):

Use LLM (Groq) to extract entities and relationships

Append entities to entity list

Append relationships to relationship list

REMOVE duplicate entities

OUTPUT: `full_graph_data` {entities, relationships}

7. Algorithms and Implementation

3. Graph Construction (graph_builder.py)

- Purpose: Build a graph in Neo4j from extracted entities and relationships.
- Key Functions:
 1. build_graph(data) → Create nodes (Entity) and relationships (RELATED) in Neo4j.
 2. clear_database() → Delete all nodes and relationships (optional fresh start).
- Output: Populated Neo4j Knowledge Graph.

Algorithm / Pseudocode

Graph Construction:
Connect to Neo4j
IF clear_db selected:
 Delete all nodes and relationships
FOR each entity:
 CREATE or MERGE Node
FOR each relationship:
 CREATE or MERGE Edge (source -> target)
CLOSE Neo4j session

7. Algorithms and Implementation

4. Graph Embeddings (graph_embeddings.py)

- Purpose: Generate vector embeddings for graph nodes to enable similarity search.
- Key Functions:
 1. generate_and_store_embeddings() → Encode nodes using SentenceTransformer and store embeddings in Neo4j.
 2. Create vector index in Neo4j for fast nearest-neighbor queries.
- Output: Nodes with embeddings + vector index for retrieval.

Algorithm / Pseudocode

Graph Embeddings:
FETCH all nodes from Neo4j
GENERATE vector embeddings for nodes using SentenceTransformer
STORE embeddings in corresponding nodes
CREATE vector index in Neo4j for fast similarity search

7. Algorithms and Implementation

5. Query Engine (query_engine.py)

- Purpose: Handle question answering and analytics over the Knowledge Graph.
- Key Functions: `search_graph(query)` → Retrieve top relevant nodes and relationships.
 - a. `get_visualization_data(query)` → Fetch nodes and edges for graph visualization.
 - b. `get_graph_analytics()` → Compute hubs and communities.
 - c. `answer_question(query)` → Generate structured answer using graph context and LLM.
- Output: Answer text, nodes & edges for visualization, analytics tables.

Algorithm / Pseudocode

Query Engine:
RECEIVE user query
ENCODE query to vector
SEARCH top-K nearest nodes in vector index
RETRIEVE neighbors and relationships
GENERATE structured answer using LLM
RETURN answer, nodes, and edges for visualization

8. Testing Integration

- Functional Testing: The system was tested by uploading multiple documents and verifying whether the document ingestion, entity extraction, knowledge graph construction, and embedding generation modules worked correctly.
- Edge Case Testing
 1. Unrelated Queries: Questions not in documents return: "The answer is not present in the knowledge graph"; no hallucinations observed.
 2. Empty Graph / Missing Nodes: System handles queries gracefully when no documents are uploaded.
 3. Duplicate / Conflicting Data: Multiple documents with same entities merge nodes; edges remain consistent.
- Cross-Document Retrieval Testing: Queries requiring information from multiple documents were tested to validate the system's ability to perform knowledge graph traversal and cross-document contextual retrieval.

9.Results and Discussion

Type	Metric	MemoryGraph AI	Vector RAG	Winner	What It Measures
Retrieval	MRR	0.787	0.787	Tie	How early a relevant entity appears (rank of first hit)
Retrieval	HitRate@K	1	1	Tie	Binary: did at least one relevant entity appear in top-K?
Retrieval	EntityCoverage	0.954	0.467	MemoryGraph AI	Fraction of all relevant entities found anywhere in the retrieved list
LLM-Judge: Answer	Relevance [1–5]	4.22	4.22	Tie	Does the answer directly address what was asked?
LLM-Judge: Answer	Faithfulness [1–5]	3.89	3	MemoryGraph AI	Is the answer factually grounded with no hallucinated details?
LLM-Judge: Answer	Completeness [1–5]	3.78	3	MemoryGraph AI	Does the answer cover all key aspects of the question?
LLM-Judge: Context	Sufficiency [1–5]	4.33	3.89	MemoryGraph AI	Does the context contain enough information to answer the question?
LLM-Judge: Context	Structural Richness [1–5]	4.11	2.67	MemoryGraph AI	Does context show HOW concepts connect, not just WHICH ones exist?

9.Results and Discussion

Metric	Meaning	Value
Files Processed	Number of documents uploaded and processed in the pipeline	4
Ingestion Time	Time taken to load and prepare the documents for processing	12.8 s
Extraction Time	Time required to extract entities and relationships from the documents to build the knowledge graph	448.41 s
Embedding Time	Time taken to generate vector embeddings for document chunks	58.76 s
Total Runtime	Overall time taken for the entire pipeline execution (ingestion + extraction + embedding)	520.04 s
Query Latency	Time taken by the system to generate an answer for a user query after processing	0.78 s

10.Publication Status

- **Title of Article:** MemoryGraph AI: Context-Aware Hybrid Knowledge Graphs with Vector-Based Analysis for Documents
- **Target Journal/Conference:** Springer- Complex and Intelligent Systems Journal
- **Current Status:** Submitted and received acknowledgement

11. Conclusions & Future Enhancement

Conclusion

- Developed a persistent, document-driven Knowledge Graph system for structured knowledge extraction and retrieval.
- Integrated entity–relationship extraction, Neo4j graph storage, and hybrid retrieval (vector + graph traversal).
- Enables cross-document contextual answering using graph-guided exploration.
- Provides interactive features like conversational history, graph visualization for every query, knowledge discovery and graph evidence for better understanding of document relationships.

11. Conclusions & Future Enhancement

Future enhancement

- Large-Scale Graph Optimization: Implement efficient indexing and caching techniques to support very large document collections and dense knowledge graphs while maintaining fast retrieval.
- Explicit Relationship Type Detection : Extend the entity–relation extraction module to identify and classify relationship types instead of storing generic “related” connections.
- Multi-Modal Knowledge Integration: Extend the system to extract knowledge from tables, figures, and diagrams in addition to text documents.

12. References

- [1] Z. Liu, M. Wu, X. Li, Y. Yan, S. Wang, C. Yang, M. Yu, Z. Zeng and M. Sun, “Graph-Anchored Knowledge Indexing for Retrieval-Augmented Generation,” arXiv preprint arXiv:2601.16462, 2026.
- [2] Z. Zhou, G. Cui, Z. Zhang, C. Yang, Z. Liu, L. Wang, C. Li and M. Sun, “MegaRAG: Multimodal Knowledge Graph-Based Retrieval Augmented Generation,” arXiv preprint arXiv:2512.20626, 2025
- [3] X. Zhu, Y. Xie, Y. Liu, Y. Li and W. Hu, “Knowledge Graph-Guided Retrieval Augmented Generation (KG²RAG),” arXiv preprint arXiv:2502.06864, 2025.
- [4] Z. Shen, C. Diao, P. Vougiouklis, P. Merita, S. Piramanayagam, D. Graux, D. Tu, Z. Jiang, R. Lai, Y. Ren and J. Z. Pan, “GeAR: Graph-Enhanced Agent for Retrieval-Augmented Generation,” arXiv preprint arXiv:2412.18431, 2024.
- [5] S. Choi and Y. Jung, “Knowledge Graph Construction: Extraction, Learning, and Evaluation,” arXiv preprint, 2024.

12. References

- [6] Q. Sun, Y. Luo, W. Zhang, S. Li, J. Li, K. Niu, X. Kong and W. Liu, “Docs2KG: Unified Knowledge Graph Construction from Heterogeneous Documents Assisted by Large Language Models,” arXiv preprint arXiv:2406.02962, 2024.
- [7] S. Yu, T. He and J. Glass, “AutoKG: Constructing Virtual Knowledge Graphs from Unstructured Documents for Question Answering,” arXiv preprint arXiv:2008.08995, 2020.
- [8] H. Zhang, J. Si, G. Yan, B. Qi, P. Cai, S. Mao, D. Wang and B. Shi, “RAKG: Document-level Retrieval Augmented Knowledge Graph Construction,” arXiv preprint arXiv:2504.09823, 2025.
- [9] L. Zhong, J. Wu, Q. Li, H. Peng and X. Wu, “A Comprehensive Survey on Automatic Knowledge Graph Construction,” arXiv preprint arXiv:2302.05019, 2023.
- [10] L. Yao, C. Mao and Y. Luo, “KG-BERT: BERT for Knowledge Graph Completion,” arXiv preprint arXiv:1909.03193, 2019.

12. References

- [11] B. Sarmah, B. Hall, R. Rao, S. Patel, S. Pasquali and D. Mehta, “HybridRAG: Integrating Knowledge Graphs and Vector Retrieval Augmented Generation for Efficient Information Extraction,” arXiv preprint arXiv:2408.04948, 2024.**
- [12] Y. Zhao, J. Zhu, Y. Guo, K. He and X. Li, “E2GraphRAG: Streamlining Graph-based RAG for High Efficiency and Effectiveness,” arXiv preprint, 2024**
- [13] P. Lewis, E. Perez, A. Piktus, F. Petroni and V. Karpukhin, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” Advances in Neural Information Processing Systems (NeurIPS), 2020.**
- [14] C. Ma, Y. Chen, T. Wu, A. Khan and H. Wang, “Large Language Models Meet Knowledge Graphs for Question Answering: Synthesis and Opportunities,” arXiv preprint arXiv:2505.20099, 2025.**
- [15] C. Mavromatis and G. Karypis, “GNN-RAG: Graph Neural Retrieval for Large Language Model Reasoning,” arXiv preprint arXiv:2405.20139, 2024.**

Thank You